STUDY GUIDE

Angular Notes

Core concepts explained in simple English

Angular 16 / 17 • Components, Routing, Forms, RxJS, NgRx & more
Rewritten from class notes • Based on the SahoSoft Angular series

Contents

1. Setup & Requirements
2. Component Lifecycle Hooks
3. The Compiler (JIT & AOT)
4. Node.js, NPM & Angular CLI
5. Library vs Framework
6. AngularJS vs Modern Angular
7. Building Blocks of Angular
8. Modules
9. Components & Templates
10. Services & Dependency Injection
11. Decorators
12. HostListener & HostBinding
13. Config Files
14. Providers & ViewProviders
15. providedIn & Tree Shaking
16. View Encapsulation
17. Input & Output
18. Directives
19. Data Binding
20. Pipes
21. Routing
22. Route Guards
23. Router Outlet & Links
24. Signals & Change Detection
25. ng-container & ng-template
26. ViewChild & Template Variables
27. Bootstrap & Angular Material
28. Forms
29. Observables, HttpClient & Promises
30. RxJS Operators
31. Interceptors
32. Session Management & Storage
33. Booting Process & Folder Structure
34. CLI Command Cheat Sheet
35. SSR & Hydration
36. Authentication: JWT & Bearer Tokens
37. Spread Operator
38. NgRx & Redux Pattern
39. NgZone & runOutsideAngular
40. Polyfills, Chunks & Loading
41. Bundling, Webpack & Vite
42. Naming Conventions
43. Platform Checks & API Security
44. Cloudflare, CDN & DNS

1. Setup & Requirements

What you need installed before you can build an Angular app.

- **IDE** — a code editor such as VS Code to write your code.
- **Node.js** — a runtime that runs JavaScript outside the browser. When you install Node.js, NPM is installed automatically.
- **NPM (Node Package Manager)** — installs and manages packages.
- **Angular CLI** — the Command Line Interface used to create and manage Angular apps.

> **IN SHORT**
> Node.js runs the tools. Angular CLI (which runs on Node.js) is what you use to create components, services, modules, and routes.

2. Component Lifecycle Hooks

Angular calls these methods automatically at different stages of a component's life. They run in this order:

Hook	When it runs
`constructor`	Runs first, as soon as the component loads.
`ngOnChanges`	Runs every time an `@Input` value changes. It receives a `SimpleChanges` object that holds the current and previous values.
`ngOnInit`	Runs once, right after the constructor, when the component loads for the first time. Good place to fetch initial data.
`ngDoCheck`	Runs on every change detection cycle — any click, text change, or activity in the component triggers it.
`ngAfterContentInit`	Runs once when projected content (content passed in from outside) is initialised.
`ngAfterContentChecked`	Runs every time the projected content is checked for changes.
`ngAfterViewInit`	Runs once when the component's own view is fully initialised.
`ngAfterViewChecked`	Runs every time the view is checked. It runs after the component is fully rendered on screen.
`ngOnDestroy`	Runs once when the component is removed from the DOM. Use it to clean up so services or timers do not keep running in the background.

3. The Compiler (JIT & AOT)

A **compiler** is a tool that translates your code into something the browser can understand and run.

You write Angular code in **TypeScript, HTML templates, and CSS**. But browsers only understand plain JavaScript, so Angular must compile (convert) your code first.

Angular 17 uses a modern build setup based on **Vite + esbuild**, which is faster than the old Webpack-based builds. It uses two compilation modes:

Mode	Used for	Details
JIT (Just-In-Time)	Development	Runs with `ng serve`. Gives fast rebuilds and live reload. Not optimised. Happens every time you refresh.
AOT (Ahead-Of-Time)	Production	Runs with `ng build --configuration production`. Produces optimised JavaScript at build time, giving a faster app and a smaller bundle.

> **REMEMBER**
> Node.js runs the build tool, but Node.js does **not** compile your Angular code — it only supports the tools that do. The final output is created inside the `dist/` folder.

4. Node.js, NPM & Angular CLI

Node.js

Node.js is a runtime environment that executes JavaScript code. It is open-source and cross-platform. We use it to build and run Angular apps, and to check versions and install packages from the command line.

NPM (Node Package Manager)

NPM installs and manages all your packages. Your main packages are listed in `package.json`, and the exact versions of those packages and their dependencies are locked in `package-lock.json`.

Angular CLI

The CLI lets you create components, services, modules, and routes by typing commands. You *can* create all of these manually, but the CLI is much faster.

- It only works properly when Node.js and NPM are installed correctly.
- Install it globally with: `npm install -g @angular/cli@latest`

5. Library vs Framework

Library	Framework
You decide the structure. You call the library when you need it and put the pieces together yourself.	The framework gives you a ready-made structure. It creates the default files, components, and routing for you.

Import & Export

- **import** — used to bring in a class or library you want to use in a component.
- **export** — used to make a class or DTO available so it can be reused elsewhere in the project.

6. AngularJS vs Modern Angular

- **AngularJS** — the original JavaScript framework by Google (2010). It is used to build single-page applications (SPAs). It uses **directives** (special HTML attributes like `ng-model`, `ng-repeat`, `ng-if`) and follows the MVC / MVVM pattern.
- **Angular (2 and above)** — the modern, TypeScript-based framework (2016 onwards). Angular 16 / 17 are versions of this.

Feature	AngularJS	Angular 17
Language	JavaScript	TypeScript
Structure	Model-View-Controller & `$scope`	Components & Services
Data Binding	Two-way binding (slower)	One-way, two-way + Signals (faster, efficient)

Quick definitions

- **JavaScript** — a scripting language.
- **TypeScript** — a superset of JavaScript. It follows OOP concepts and catches errors at compile time.

Why Angular?

- It is a framework that uses TypeScript.
- It builds single-page applications.
- It is open-source and cross-platform (web, mobile, desktop).

7. Building Blocks of Angular

The main building blocks are: **Modules, Components, Templates, Metadata, Data Binding, Directives, Services,** and **Dependency Injection**. Each is explained in the sections that follow.

8. Modules

A module is a class decorated with `@NgModule`. Every Angular application needs at least one module, and one module can contain many components.

- **Root module** — the main parent module (`AppModule`).
- **Feature module** — a child module for a specific feature.

> **EXAMPLE**
> Think of a college:
> • **Account module** → fees deposit, credit, check status, pending, submit (these are components).
> • **Study module** → syllabus, exam paper, results, holidays (these are components).

Inside @NgModule

- **declarations** — the components that belong to this module.
- **imports** — other modules this module needs.
- **providers** — services, classes, and pipes.
- **bootstrap** — the first component to load (the start of the app).

9. Components & Templates

Component

A component is a class where you write your logic. It works like a controller.

- Every component has its own **selector**, which is registered in the HTML.
- Services are injected through the constructor before they are used.
- Components are reusable.
- The `@Component()` decorator (from Angular's core library) sits above the class and marks it as a component.

Template

The template is the HTML that belongs to a component. There are two ways to write it:

- **template** — write the HTML inline, inside the component file itself.
- **templateUrl** — point to an external HTML file using a path.

10. Services & Dependency Injection

Service

Write code once in a service, then reuse it anywhere instead of repeating it. For example, code that calls an API can live in a service and be used by many components.

Create one with: `ng generate service myServiceName`

Dependency Injection (DI)

When one thing depends on another, that is dependency injection. A component often needs a service (for example, to call an API), so the service is a dependency. Angular creates and supplies (injects) that service for you.

Injecting a service

You inject a service through the constructor, then use it:

```
// Inject
constructor(private _exampleService: ExampleService) {}

// Use
ngOnInit() {
  this.title = this._exampleService.someMethod();
}
```

@Injectable()

Use `@Injectable()` when you want to inject one service inside another service (a nested service). Without it, you cannot use dependency injection at that level.

Singleton vs Multiple instances

Singleton (one shared instance)	Multiple instances
One object is shared across the app. Saves memory and is faster to create. Shares state everywhere.	Each component gets its own isolated object and its own state. Uses more memory if overused.

> **EASY ANALOGY**
> **Singleton** = four friends open one company with **one** shared bank account. If one person adds or removes money, all four see the change.
> **Multiple** = four friends, but **four separate** accounts. A change in one account does not affect the others.

These objects live in the browser's RAM and are managed by the JavaScript engine (for example, V8 in Chrome).

11. Decorators

A decorator is a TypeScript feature written with a `@` prefix and parentheses, for example `@Component()` or `@NgModule()`. It tells Angular what a class is.

- The settings written inside a decorator are called **metadata** (for example `selector`, `template`, `templateUrl`).
- Decorators run at runtime.
- Without a decorator, Angular cannot tell whether a class is a component, module, or service.
- A decorator must always sit directly above its class.

Common decorators

Decorator	Purpose
`@NgModule`	Defines a module
`@Component`	Defines a component
`@Injectable`	Defines a service
`@Input` / `@Output`	Send and receive data between components

Types of decorator

- **Class decorator** — `@Component`, `@NgModule`.
- **Property decorator** — `@Input`, `@Output`.
- **Method decorator** — `@HostListener()` (reacts to events like click, mouseover, dblclick).
- **Parameter decorator** — `@Inject` (used inside the constructor).

Note: Each decorator has its own unique role.

12. HostListener & HostBinding

`@HostListener` is a method decorator used to listen for browser events on the component's host element — events like click, mouseover, mouseleave, or double-click.

```
@HostListener('click') onClick() {
  alert('clicked');
}
```

REAL EXAMPLE

An interactive map of India: when you hover over a state, some value appears. That hover behaviour can be handled with `@HostListener`.

13. Config Files

File	What it does
`app.config.ts`	Configuration file where you register things like interceptors. It works at the application level. Both `AppConfig` and `AppComponent` are loaded from `main.ts`.
`web.config`	Server-level configuration (for IIS). Used for security and rules such as forcing lowercase URLs, 301 redirects, and HTTP-to-HTTPS.
`angular.json`	Main project configuration. Defines the web root, build settings for dev/production, and third-party paths (for example, adding Bootstrap's CSS).
`environment.ts`	Holds environment settings (similar to an app-settings file). There is usually one for development and one for production.

Other terms

- **Constructor** — loads automatically and first when a component loads.
- **camelCase** — first word lowercase, later words capitalised, e.g. `camelCase`, `sunilKulshrestha`.
- **preserveWhitespace** — metadata (boolean, default `false`) that keeps the spacing between elements, e.g. spacing between four buttons placed together.

14. Providers & ViewProviders

providers is metadata used to register (inject) a service. You can register it at the module level or the component level.

Module level	Component level
One **single shared object** (singleton) is created and reflected in all components. The drawback: even components that do not use the service still get it.	A **separate object** is created for each component that injects it. Only that component is affected.

Syntax: `providers: [MessageService]`. Without registering a service in providers, it cannot load.

ViewProviders

`viewProviders` is metadata used when you want to inject a class directly into a component (rather than the module). Use `providers[]` at the module level and `viewProviders[]` at the component level. It is mainly used to give access to a class or pipe inside the component's view.

15. providedIn & Tree Shaking

providedIn (added in Angular 6)

Use `providedIn` when you want a singleton service **without** registering it at the module level. You write it inside the service's `@Injectable`:

```
@Injectable({ providedIn: 'root' })
```

- You do not need to add it to a `providers` array.
- It is loaded at runtime, only when a component needs it.
- It makes the service globally available and creates only one shared instance (singleton).

Before Angular 5 you had to use `@Injectable` and then register the service at the module or component level.

Tree Shaking

Tree shaking means removing unused code. When Angular builds the bundle, code that is never used is dropped.

For example, if you register two API services in providers but use only one, tree shaking removes the unused one from the bundle. This works with `providedIn` because the service is not tied to a providers array and is loaded at runtime.

16. View Encapsulation

Encapsulation is metadata that controls how far a component's CSS reaches. There are three modes:

Mode	Behaviour
Emulated (default)	No Shadow DOM, but styles are scoped. Styles do not leak from parent to child or child to parent.
ShadowDom (Native)	Uses real Shadow DOM. Parent styles can reach the child, but child styles do not reach the parent.
None	No Shadow DOM and no scoping. Styles flow freely both ways — parent to child and child to parent.

```
encapsulation: ViewEncapsulation.None;
```

17. Input & Output

`@Input`	`@Output`
Sends data **from parent to child**. The child receives data from the parent.	Sends data **from child to parent**. The parent receives data from the child using `EventEmitter` and `emit()`.

Input example

```
@Input() moonsignWidget: any;

<app-baby-name-widget [moonsignWidget]="data?.knowYourMoonsign"></app-baby-name-widget>
```

Output example

```
@Output() isPageChanged = new EventEmitter<number>();
this.isPageChanged.emit(this.currentPage);

<app-pagination (isPageChanged)="onPageChange($event)"></app-pagination>

onPageChange(data: any) {
  this.router.navigate(['blog', data]);
}
```

Tip: `any` as a type means "accept any data type here".

18. Directives

A directive changes the appearance or behaviour of a DOM element. There are three types:

Type	What it does
Component directive	A component used inside another component that changes based on a condition.
Structural directive	Changes the structure of the DOM (adds or removes elements). Examples: `*ngIf`, `*ngFor`, `*ngSwitch`. These always use the `*` (asterisk).
Attribute directive	Changes appearance dynamically, e.g. background colour or font size.

Create one with: `ng g directive changeColour`. Directives are good for functionality you want to reuse globally, such as a button timer.

*ngIf

Adds or removes an element from the DOM based on a condition.

```
<div *ngIf="condition"></div>

<!-- with else -->
<div *ngIf="condition; else elseBlock"></div>
<ng-template #elseBlock></ng-template>

<!-- with then and else -->
<div *ngIf="condition; then thenBlock else elseBlock"></div>
<ng-template #thenBlock></ng-template>
<ng-template #elseBlock></ng-template>
```

*ngSwitch

Uses `[ngSwitch]`, `*ngSwitchCase`, and `*ngSwitchDefault`. It is case-sensitive (uppercase and lowercase must match).

> **NGIF VS NGSWITCH**
> `*ngIf` works with a boolean (true/false). `*ngSwitch` matches a value against several cases, not just true/false.

19. Data Binding

Data binding connects your component class (`.ts`) with the view (the HTML template). There are two main kinds:

One-way binding	Two-way binding
Data flows in one direction: component → view (interpolation, property, style, class, attribute) or view → component (events).	Data flows both ways: a change in the `.ts` file updates the HTML, and a change in the HTML updates the `.ts` file.

One-way: component to view

- **Interpolation** — `{{ propertyName }}`
- **Property binding** — `<span [innerHTML]="firstName"></span>` (the square brackets `[]` make it property binding).
- **Attribute binding** — set an attribute dynamically:

```
cols: number = 3;
<td [attr.colspan]="cols" align="center">Student Details</td>
```

- **Event binding** — respond to events like click or keypress:

```
<button (click)="onClick()">Click me</button>
```

Two-way binding

Combining property binding and event binding gives two-way binding. The shortcut is `[(ngModel)]`:

```
<!-- long form -->
<input [value]="data" (input)="data = $event.target.value">
Your name: {{ data }}

<!-- short form -->
<input [(ngModel)]="data">
```

20. Pipes

A pipe (written with `|`) transforms data for display in the template. It changes the **format** of the data, not the original value. For example, showing lowercase input as uppercase.

```
{{ 'Test' | uppercase }}   <!-- Output: "TEST" -->
```

There are two kinds: **built-in pipes** and **custom pipes**.

Built-in pipes

Common ones: `date`, `uppercase`, `lowercase`, `currency`, `percent`, `number`, `json`, `slice`, `titlecase`, `async`.

- **Parameterized pipe** — a pipe that accepts parameters:

```
{{ value | pipeName : param1 : param2 }}
{{ salary | currency:'USD':true }}   // $123456
{{ dob | date:'medium' }}
```

- **Chaining pipes** — applying more than one pipe to the same data:

```
{{ salary | currency:'USD':true | uppercase }}
```

- **JSON pipe** — shows data in JSON format: `{{ countries | json }}`

Custom pipes

When the built-in pipes are not enough, create your own with your own logic: `ng g pipe myPipe --flat`

```
// reverse.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'reverse' })
export class ReversePipe implements PipeTransform {
  transform(value: string): string {
    return value.split('').reverse().join('');
  }
}

// Use:  {{ 'hello' | reverse }}   --> 'olleh'
```

Pure vs Impure pipes

Pure pipe (default)	Impure pipe
Runs **only when the input value changes**. If the value stays the same, the pipe does not run again.	Runs **every time** Angular checks the page, even if the value has not changed. Set with `pure: false`.

```
@Pipe({ name: 'double', pure: false })  // impure
export class DoublePipe {
  transform(value: number): number { return value * 2; }
}
```

21. Routing

Routing lets you navigate between pages **without refreshing** the browser. It moves between components (not HTML pages), which is what makes a single-page application feel fast. The Router is a library.

How the Router works (7 steps)

1. **Parse the URL** — read the URL and build a router tree. Special characters:
   - `/` divides URL segments
   - `( )` secondary routes
   - `:` named router outlet
   - `;` matrix parameter
   - `?` query string parameters
   - `#` fragment
   - `//` multiple secondary routes
2. **Redirect** — apply any redirects. Two kinds:
   - **Local redirect** (no leading slash): `{ path: 'one', redirectTo: 'two' }`
   - **Absolute redirect** (leading slash, replaces whole URL): `{ path: 'one', redirectTo: '/two' }`
3. **Identify the router state** — using the router tree from step 1.
4. **Run guards** — check authentication / authorization before the user can proceed.
5. **Run resolvers** — fetch the data the route needs.
6. **Activate** — activate the component and show the page.
7. **Manage** — handle the navigation and repeat the process when a new URL is requested.

22. Route Guards

A guard decides whether a user is allowed to access a route. For example, if data is missing it can stop navigation, or it can save an unsaved form when the user accidentally leaves.

- **Authentication** — is the user logged in or not?
- **Authorization** — after login, which areas is the user allowed to access?

Create a guard with: `ng g guard auth`

The five guards

Guard	Purpose
CanActivate	Checks if the user can enter a route (login / access check).
CanActivateChild	Same as CanActivate, but for child routes.
CanDeactivate	Fires when leaving a page. Example: a "Are you sure you want to leave?" popup if a form is half-filled.
Resolve	Pre-fetches the data a route needs before the component loads.
CanLoad	Used with lazy loading (`loadChildren`) to allow or block a module from loading.

```
{ path: 'aboutus', component: AboutComponent, canActivate: [auth] }
```

CanActivate vs CanLoad

CanLoad	CanActivate
Runs **before** the module is loaded. If blocked, the module's JavaScript is never downloaded. Saves time, bandwidth, and prevents unauthorized access. (The guard at the front gate.)	Runs **after** the module is already downloaded. It decides if the user can enter. (The guard at the room door.)

23. Router Outlet & Links

Router Outlet

`<router-outlet>` is the placeholder where the routed component is displayed. When you hit a URL, the matching component appears at this position. Without a router outlet, components and modules will not show.

Auxiliary (named) routes

When you use multiple outlets, you give the extra ones a name:

```
<router-outlet></router-outlet>                    <!-- primary -->
<router-outlet name="booklist"></router-outlet>   <!-- secondary / auxiliary -->
```

Router Link

```
<!-- Static link -->
<a [routerLink]="['/student']">Student</a>

<!-- Dynamic link (from the .ts file) -->
<button (click)="student()">Show student</button>

export class AppComponent {
  constructor(private router: Router) {}
  student() { this.router.navigate(['/student']); }
}
```

Default, wildcard & parameterized routes

```
// Load a component by default
{ path: '', redirectTo: 'student', pathMatch: 'full' }

// Normal route
{ path: 'student', component: StudentComponent }

// Wildcard (404) - keep this LAST
{ path: '**', component: PageNotFoundComponent }
```

Parameterized routes have two styles:

Snapshot	Observable / Stream
Reads the route value once. If data does not arrive the first time, refreshing the page will not bring it, because the snapshot is fixed.	Keeps listening. If data is missing the first time, a refresh can deliver it through `subscribe`.

Use `param` for a single parameter and `paramMap` for multiple parameters.

24. Signals & Change Detection

Signals

A signal updates only the specific part of the DOM that changed, instead of comparing the whole DOM. It works a bit like the virtual DOM idea. It is useful when data does not update in real time with the normal flow.

```
// Signal syntax
ylGiftingData = signal<any>(null);
this.ylGiftingData.set(userData);

effect(() => {
  this.pusherAnimationUrl = this.pusher.ylGiftingData().AnimationUrl;
});
```

Change Detection

Change detection is how Angular automatically refreshes the view when data changes.

Signals	Change Detection
Updates only the specific DOM part that changed. Does not compare the whole DOM.	Compares the whole DOM, then updates the part that changed.

ChangeDetectorRef is a service that lets you control this manually — you can tell Angular when to check or update the view.

Subject (for cross-component messaging)

```
private messageSubject = new Subject<ChatPayload>();
message$ = this.messageSubject.asObservable();

sendMessageToLiveChat(message: string, gifUrl: string | null, duration: number | null) {
  this.messageSubject.next({ message, gifUrl, duration });
}

this.yogiLiveService.message$.subscribe((data) => {
  this.sndMessageViaApi = data.message;
  this.pusherGifUrl = data.gifUrl ?? '';
});
```

25. ng-container & ng-template

ng-container

`<ng-container>` is used to render HTML **without adding an extra tag** to the DOM. It shows the content but not a wrapper element.

- You can use structural directives on it (it is optional), e.g. `*ngIf`.
- Handy when you do not want an extra `<div>`.

ng-template

`<ng-template>` renders HTML and is used with conditions like `ngIf`, `ngFor`, or `ngSwitch`.

- It shows content but not a tag — its tag is automatically commented out and not added to the DOM. Only the content is added.
- You **must** use a structural directive with it.
- CSS cannot be applied directly to it (unlike a `<div>`).
- Do not write the `*` shorthand on it — the `*` is itself shorthand for `ng-template`, so it would cause an error.

```
<ng-template [ngIf]="display == 'true'">show with template</ng-template>
```

26. ViewChild & Template Variables

ViewChild

`@ViewChild()` is a decorator that lets a parent component access a method, property, or element from its child. You can use it to reach:

- **A child component**:

```
@ViewChild(CounterComponent) counterComponent: CounterComponent;
```

- **A directive** — access a method on a directive (the directive acts like a child).
- **A DOM element** via a template reference variable:

```
<input type="text" #name>
@ViewChild('name') elName: ElementRef;
```

Related decorators: `ViewChildren`, `ContentChild`, `ContentChildren`.

Template reference variable

A variable declared in the HTML using `#`. It lets you read a DOM value or pass it to a method.

```
<input #nameInput type="text">
<button (click)="greet(nameInput.value)">Greet</button>

greet(name: string) {
  console.log('Hello,', name);
}
```

27. Bootstrap & Angular Material

Bootstrap

Bootstrap is an open-source CSS framework that gives you ready-made HTML, CSS, and JavaScript (buttons, calendars, etc.) so you do not need separate third-party libraries.

```
npm install bootstrap --save
```

Then register it in `angular.json` under styles:

```
"../node_modules/bootstrap/dist/css/bootstrap.min.css"
```

It provides utility classes like `btn`, `btn-primary`, `btn-danger`, `btn-lg`.

Angular Material

Angular Material is a UI component library. You can copy ready-made components from material.angular.io.

```
npm install --save @angular/material @angular/cdk @angular/animations
```

28. Forms

Angular has two kinds of forms:

Template-driven	Reactive (Model-driven)
Simple forms (login, registration). Most of the work is in the HTML template; very little in the component. Import `FormsModule`. Best for small forms.	For large forms. Most of the work is in the component (`.ts` file); little in the HTML. Import `ReactiveFormsModule`.

Template-driven form

```
<form #regForm="ngForm" (ngSubmit)="register(regForm)">
```

- `regForm.value` — get the form's values.
- `regForm.valid` — check if the form is valid.
- `regForm.touched` — check if the user has visited a control.

Reactive form

You need three classes from the forms library: **FormGroup**, **FormControl**, and **FormBuilder**.

- **FormControl** — tracks and validates a single control (one input box).
- **FormGroup** — tracks the validity and state of a group of controls. All the form data is stored here.
- **FormBuilder** — a helper that groups all controls and lets you add validation to each field.

```
this.signupForm = formBuilder.group({
  firstName: ['', Validators.required]
});

// Multiple validators on one control with compose:
firstName: [null, Validators.compose([
  Validators.required,
  Validators.minLength(10),
  Validators.maxLength(200)
])]
```

Setting and watching values

Method	Use
`setValue()`	Set the value of **all** controls.
`patchValue()`	Set the value of **some** controls only.
`valueChanges`	Get a control's value in real time whenever it changes.
`statusChanges`	Fires during validation and reports valid / invalid status.
`FormArray` / `push`	Add multiple values or new controls to the form dynamically.

```
this.signupForm.get('firstName').valueChanges
  .subscribe(uname => console.log(uname));
```

29. Observables, HttpClient & Promises

Observable

An Observable is like a **stream of data over time** that you can watch (subscribe to) and react to. Think of it as a live feed: every time something new happens, you get an update. Angular uses Observables (from RxJS) to handle asynchronous work like API calls and user events.

```
const data$ = of('Apple', 'Banana', 'Mango');
// data$ is an Observable. The $ is just a naming convention.
// It will emit 'Apple', 'Banana', 'Mango' to anyone who subscribes.
```

- Install RxJS to use Observables: `npm install rxjs --save`
- **subscribe** — how you receive the values from an Observable.
- **emit** — means an Observable sends out a value.
- **| async** — an Angular pipe that automatically subscribes to an Observable and updates the view when data arrives. Often used with `*ngIf`.

HttpClient

`HttpClient` is the Angular service for making HTTP requests (get, post, put, delete). It is built on top of Observables, so each request returns an Observable you subscribe to. Import `BrowserModule` first, then `HttpClientModule`.

```
getBook(): Observable<Book[]> {
  return this.http.get<Book[]>('api-url');
}
```

Main methods: `get`, `post`, `put`, `delete`, `patch`, `head`, `options`.

POST parts

- **url** — the API endpoint.
- **body** — the data you want to send.
- **options** — optional headers and parameters. A header carries information about the data so the server can identify it easily.

```
return this.http.post<Book[]>('api-url', book, options);
```

Promise

A Promise handles something that takes time (like fetching data) **without freezing the app**. It returns a single value. Without it, the app would pause and wait. Use it when working with some third-party libraries or one-time delayed data.

> **OBSERVABLE VS PROMISE**
> An **Observable** can emit many values over time and can be cancelled. A **Promise** returns a single value and cannot be cancelled.

In-Memory Web API

A tool to create a fake API inside Angular for testing:

```
npm install angular-in-memory-web-api@0.5.3 --save
```

30. RxJS Operators

RxJS (Reactive Extensions for JavaScript) is a library for handling asynchronous and event-based code with Observables. You chain operators inside `.pipe()` to transform or control the data stream.

Operator	What it does
`of()`	Creates an Observable from given values. `of(1,2,3)`
`map()`	Transforms each value. `map(v => v * 2)` → 2, 4, 6
`filter()`	Keeps only values that pass a condition. `filter(v => v % 2 === 0)`
`mergeMap()`	Runs all inner Observables in parallel and returns results as they come.
`switchMap()`	Cancels the previous inner Observable when a new value arrives. Good for search inputs — only the latest matters.
`concatMap()`	Runs inner Observables one after another, in order. Waits for the previous to finish.
`exhaustMap()`	Ignores new values while the current one is still processing. Good for ignoring rapid repeat clicks.
`debounceTime()`	Waits for a quiet period before emitting the latest value. Great for search boxes so you do not hit the API on every keypress.
`forkJoin()`	Waits for all Observables to finish, then emits all results together as an array.
`catchError()`	Handles errors in the stream and can recover with a fallback value.
`retry()`	Retries the Observable a set number of times if it errors.

debounceTime vs setTimeout

debounceTime()	setTimeout
From RxJS, used with streams. Handles multiple rapid inputs automatically and resets on its own. No manual cleanup needed.	A plain JavaScript function. You must handle everything manually, including `clearTimeout()`.

```
fromEvent(document, 'keyup')
  .pipe(debounceTime(300))
  .subscribe(() => console.log('Stopped typing...'));
```

31. Interceptors

An interceptor lets you add or change something on every HTTP request and response — for example, attaching a bearer token to each request, or handling a 500 error globally. It lives inside the HttpClient module.

```
// auth.interceptor.ts
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private auth: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = this.auth.getToken();
    if (token) {
      const cloned = req.clone({
        headers: req.headers.set('Authorization', `Bearer ${token}`)
      });
      return next.handle(cloned);
    }
    return next.handle(req);
  }
}

// Register it in the module
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
]
```

32. Session Management & Storage

Storage can be on the client (browser) or the server.

Client-side storage

Type	Details
Cookies	Small storage (about 5 MB limit). Install `ngx-cookie-service`.
localStorage	No expiry — data stays until the user clears it. Never sent to the server. (~10 MB)
sessionStorage	Cleared when the browser tab is closed. Otherwise same as localStorage. Good for things like net-banking sessions.

```
// Cookies
this.cookie.set('userid', 'ABC');   // key, value
this.cookie.get('userid');

// localStorage
localStorage.setItem('key', value);
localStorage.getItem('key');
localStorage.removeItem('key');
localStorage.clear();

// Storing an object as JSON
let obj = { fName: 'Sunil', lName: 'Kumar' };
localStorage.setItem('user', JSON.stringify(obj));   // object -> string
JSON.parse(localStorage.getItem('user'));            // string -> object
```

- `JSON.stringify(obj)` — converts an object into a JSON string.
- `JSON.parse(str)` — converts a JSON string back into an object.

Server-side storage

Cache — data stored on the application server. For example, .NET provides `IMemoryCache` to manage cached data.

33. Booting Process & Folder Structure

How the app starts (booting)

1. `index.html` — loaded first by the browser. There is only one HTML file in the whole app.
2. `main.ts` — Angular's entry point. It bootstraps the root module or component.
3. `app.module.ts` — loads the root module (if using NgModules).
4. `app.component.ts` — the root component.
5. `app.component.html` — rendered inside the `<app-root>` tag in `index.html`.

Metadata inside @NgModule

Key	Holds
`declarations`	Components that belong to the module.
`imports`	Other modules (e.g. `BrowserModule`, `HttpClientModule`).
`providers`	Services and pipes.
`bootstrap`	The component that loads first.

Key folders & files

Item	Purpose
`node_modules`	Holds all installed packages, referenced by `package.json`.
`package.json`	Lists your main packages.
`package-lock.json`	Locks the exact versions of all dependency packages.
`assets`	Images and other static files.
`favicon.ico`	The small icon shown in the browser tab.
`.editorconfig`	Keeps coding style consistent across a team. Rules here are enforced.
`.gitignore`	Lists files Git should ignore.
`angular.json`	Project configuration (dev/prod settings, third-party paths).
`tsconfig.json`	TypeScript configuration.
`README`	Information on how to install and run the app.

Version numbers

A version like `16.1.8` means **major.minor.patch**.

- **Caret** `^` — allows updates to the last two digits (minor + patch).
- **Tilde** `~` — allows updates to the last digit (patch only).

34. CLI Command Cheat Sheet

Run all of these in the Node.js command prompt.

Command	What it does
`node -v`	Check the Node.js version.
`npm -v`	Check the NPM version.
`npm install -g @angular/cli`	Install the Angular CLI globally.
`ng v`	Check the CLI version.
`ng new myFirst`	Create a new project.
`ng serve` / `ng serve -o`	Run the app (`-o` opens the browser automatically).
`ng g c student`	Generate a component.
`ng g m account`	Generate a module.
`ng g c account/fee`	Generate a component inside a module.
`ng g class create`	Generate a class (creates `.ts` and `.spec.ts`).
`ng g module app-routing --flat --module-app`	Generate a routing module.
`ng generate service new-folder/my-service`	Generate a service.
`ng g directive changeColour`	Generate a directive.
`ng g pipe myPipe --flat`	Generate a custom pipe.
`npm install bootstrap --save`	Install Bootstrap.
`npm install jquery --save`	Install jQuery.
`npm i --legacy-peer-deps`	Install when you need to force a package update past peer-dependency conflicts.

Quotes for code

- **Single quotes** `' '` — for single-line strings.
- **Backticks** `` ` ` `` — for multi-line strings, or when mixing static text with values from an API (template literals).

35. SSR & Hydration

Server-Side Rendering (SSR)

SSR means the website's pages are built on the **server** instead of in the user's browser.

Traditional rendering	Server-side rendering
The browser downloads JavaScript, runs it to build the page, then displays it.	The server builds the full HTML first and sends a ready-to-view page to the browser.

Benefits

- **Faster load times** — users see a complete page sooner.
- **Better SEO** — search engines can read fully rendered HTML easily.
- **Better experience** — especially on slow devices or connections.

In Angular, SSR is handled by `server.ts`.

Hydration

After the server sends ready-made HTML, the page looks fine but is not interactive yet — buttons do not click and forms do not submit. **Hydration** is the step where Angular takes over that static HTML and connects it to your components and event handlers, making it interactive.

Without hydration	With hydration
Angular throws away the server HTML and rebuilds everything in the browser. Slower, uses more memory, causes a delay.	Angular keeps the server HTML and just "wakes it up". The app becomes interactive faster and uses fewer resources.

36. Authentication: JWT & Bearer Tokens

JWT (JSON Web Token)

A JWT is a **type** of token that contains data such as user ID, role, and expiry time. It looks like a long encoded string (`eyJhbGciOiJIUzI1NiIs...`). After you log in, the server gives you this token.

Bearer token

"Bearer token" is not a type of token — it is a **way of sending** a token, by placing it in the HTTP header:

```
GET /profile
Authorization: Bearer eyJhbGciOiJIUzI1NiIs...
```

It is commonly used in token-based authentication systems like OAuth 2.0 and JWT, and suits stateless APIs (no session stored on the server).

The flow in Angular

Step	What happens
Backend	Validates username & password, then issues a bearer token (e.g. a JWT).
Frontend	Stores the token in `localStorage` or memory.
Send	Attaches the token to each request via an interceptor.
Header	`Authorization: Bearer <token>`
API	Validates the token and grants (or denies, e.g. 403) access.

```
// auth service
login(credentials: any) {
  return this.http.post<any>('http://localhost:3000/login', credentials);
}
saveToken(token: string) { localStorage.setItem('token', token); }
getToken() { return localStorage.getItem('token'); }
```

37. Spread Operator

The spread operator (`...`) makes a shallow copy of an object, creating a new object with the same properties instead of just pointing to the original.

```
const pack = { id: 1, amount: 100 };
const copy = { ...pack };
console.log(pack === copy);  // false -> they are different objects
```

It is a quick way to copy all of a form's fields into a new object without writing each one by hand:

```
const formData = {
  ...this.commonForm.value,          // copies every field from the form
  userGender: this.selectedUserGender,
  partnerGender: this.selectedPartnerGender
};

// Result:
// { name: "Rahul", dob: "1990-01-01", userGender: "male", partnerGender: "female" }
```

WHY IT MATTERS

Without the spread operator you would have to copy each field manually — long and hard to maintain. Trying to drop an object inside another object without `...` gives a syntax error.

38. NgRx & Redux Pattern

Redux pattern

Redux is a way to manage application state in one central place. It has three key pieces: **Store, Action, Reducer**.

The flow:

1. The user does something (e.g. clicks "Add Task").
2. An **Action** is created: `{ type: 'ADD_TODO', payload: 'Buy milk' }`
3. The **Reducer** receives the action and updates the state.
4. The **Store** now holds the new state.
5. The UI automatically reflects the change.

Why use it? It is **predictable** (you always know how state changes), **centralised** (all state in one place), **debuggable** (track every change), and **scalable** (great for large apps).

NgRx

NgRx is a state-management library for Angular, built on RxJS and inspired by Redux. Its main parts:

Part	Role
Store	The single source of truth — all state is kept here.
Actions	Describe events, e.g. update or add something.
Reducers	Update the state based on actions.
Selectors	Read specific data from the state easily.
Effects	Handle side effects like HTTP requests.

Example: a counter

```
// State
export interface CounterState { count: number; }
export const initialState: CounterState = { count: 0 };

// Actions
export const increment = createAction('[Counter] Increment');
export const decrement = createAction('[Counter] Decrement');
export const reset     = createAction('[Counter] Reset');

// Reducer
export const counterReducer = createReducer(
  initialState,
  on(increment, (state) => ({ ...state, count: state.count + 1 })),
  on(decrement, (state) => ({ ...state, count: state.count - 1 })),
  on(reset, () => initialState)
);
```

39. NgZone & runOutsideAngular

Angular runs change detection whenever something happens (like a timer tick). For heavy or frequent background work, this can slow the app down. `NgZone.runOutsideAngular()` lets that work run **without** triggering change detection, and `ngZone.run()` lets you step back in to update the UI when needed.

```
// With runOutsideAngular
startTimer() {
  this.ngZone.runOutsideAngular(() => {
    setInterval(() => {
      this.counter++;
      if (this.counter % 5 === 0) {
        this.ngZone.run(() => {});   // update the UI every 5 seconds
      }
    }, 1000);
  });
}
```

Setup	Result
Without `runOutsideAngular`	Angular checks for updates on every interval — can be slow.
With `runOutsideAngular`	Angular ignores the background work entirely.
With both (`runOutsideAngular` + `run()`)	Background tasks do not slow the UI, and the UI updates only when needed.

40. Polyfills, Chunks & Loading

Polyfills

Angular uses modern JavaScript features (`Promise` , `fetch` , `Array.includes()` , etc.). Some old browsers (like Internet Explorer 11) do not understand them. **Polyfills** fill that gap so your app works everywhere. The file is created automatically (`src/polyfills.ts`).

Chunk files

A chunk is a smaller, separate JavaScript file holding part of your app. Angular splits code into chunks so files load only when needed — this is **lazy loading**.

Eager vs Lazy vs Preloading

Strategy	Behaviour
Eager loading	Everything (parent + children) loads at startup. No new chunk.
Lazy loading	A module loads only when the user visits it. Angular creates a chunk for it. Configured with `loadChildren` in the routing file.
Preloading	Some modules load at startup; others load on demand. First make modules lazy, then apply Angular's `PreloadingStrategy` .

```
// Lazy loading a module
{ path: 'company', loadChildren: () => import('./company/company.module') }

// Lazy loading standalone components (separate chunk each)
{ path: 'home/a', loadComponent: () => import('./home/a/a.component').then(m =>
m.AComponent) },
{ path: 'home/b', loadComponent: () => import('./home/b/b.component').then(m =>
m.BComponent) }
```

Note: If a parent template loads several children together, Angular creates one chunk for that group, not one per child.

41. Bundling, Webpack & Vite

A real app has many files — dozens of TypeScript files, CSS files, images, and templates. Sending them all separately would be slow. **Bundling** combines them into fewer, optimised files. Angular does this automatically and stores the result in the `dist/` folder.

What bundling does

- **Bundling** — combines many files into a few.
- **Minifying** — removes spaces and comments to shrink files.
- **Transpiling** — converts TypeScript / newer JS into older JS that all browsers understand.
- **Tree shaking / code splitting** — removes unused code and splits the rest into chunks that load on demand.

Webpack vs Vite

Webpack	Vite
The older tool that bundled and optimised code. Used in Angular versions before 17.	A newer, faster tool with less setup. Used by default from Angular 17. Handles all bundling and optimisation for you.

DO YOU NEED WEBPACK IN ANGULAR 17?
No. From Angular 17 the team moved to Vite. The Angular CLI (`ng serve` , `ng build --configuration production`) already does everything Webpack used to do.

42. Naming Conventions

Style	Rule	Example
PascalCase	Every word starts with a capital letter; no spaces.	`MyCustomPipe`, `AppComponent`
camelCase	First word lowercase, later words capitalised.	`myCustomPipe`, `getUserData()`
snake_case	All lowercase, words joined by underscores.	`my_custom_pipe`, `user_name`
kebab-case	All lowercase, words joined by dashes.	`<app-user-profile>`, `user-profile.component.ts`

Other quick terms

Term	Meaning
standalone: true	A standalone component does not need to be declared in an NgModule. This reduces boilerplate and is easier to maintain.
boilerplate code	Repetitive setup code that you want to reduce or remove.
DOM	The page structure the browser builds and updates.
Real (main) DOM	When data changes, the whole DOM is rebuilt and re-rendered — slower. Used by Angular.
Virtual DOM	Only the changed part is updated — faster. Used by React.

React vs Angular

Angular	React
A framework. Uses two-way binding and the real DOM.	A library. Uses one-way binding and the virtual DOM.

43. Platform Checks & API Security

isPlatformBrowser & isPlatformServer

With SSR, the same code can run on both the server and the browser. These helpers let you run code only where it is safe.

Function	Use it to…
`isPlatformBrowser(platformId)`	Run code only in the browser. Avoids running browser-only code (`window` , `localStorage` , DOM) on the server.
`isPlatformServer(platformId)`	Run code only on the server, e.g. preloading data for SEO, logging, or server-side caching.

Securing an API (.NET + Angular)

Area	Methods
Authentication	JWT, OAuth2, IdentityServer, Cookies.
Authorization	Roles, Policies, Claims.
CORS (Cross-Origin Resource Sharing)	Allow only specific origins (e.g. let your frontend domain call your backend domain).
CSRF (Cross-Site Request Forgery)	Anti-forgery tokens.

44. Cloudflare, CDN & DNS

Cloudflare

Cloudflare is **not** a server where you store files. It is a service that sits between your website and its visitors, helping your real server by protecting it, speeding up delivery, and managing traffic.

People use it for security, speed, free SSL (HTTPS), better DNS, faster load times, and caching static Angular files (like `main.js`, `runtime.js`) that rarely change. Its services include a CDN, DDoS protection, a Web Application Firewall (WAF), DNS, SSL/TLS, and edge computing (Cloudflare Workers).

CDN (Content Delivery Network)

A CDN speeds up your site by caching static content (HTML, CSS, JS) on servers around the world, so users download from a server near them.

DNS (Domain Name System)

DNS translates a human-friendly website address into a machine-friendly IP address, so your browser knows where to connect. DNS is not the IP itself — it is the system that looks it up.

```
You type:   https://astroyogi.com/
DNS returns: 13.224.123.45
Browser connects to that IP address.
```

End of notes • Angular core concepts, rewritten in simple English